WEB SYSTEMS DESIGN – PROJECT 7

VISTULA
UNIVERSITY

Name: Amjad Massaoud

Student ID: 78709

Course: Web Systems Design

Semester: 6th semester 57DPH

Date: 30.08.2026

1. Introduction

The purpose of this laboratory project is to extend the existing production-like web system with a new architectural module: restaurant proximity search. In previous laboratories, the system already included a Laravel backend, Vue frontend, PostgreSQL database, Redis cache, Docker Compose deployment, Nginx reverse proxy, API versioning, a task API, and a URL shortener module.

Project 7 focuses on geolocation and read-heavy search. Restaurants are stored with latitude and longitude values. A user can provide current coordinates and a radius, and the backend calculates which restaurants are nearby. The response is returned as structured JSON and sorted by distance.

This type of system is important in real web applications such as food delivery platforms, ride-hailing services, local recommendations, and map-based search. Even though this laboratory uses a simplified implementation, it demonstrates important architectural topics: coordinate storage, distance calculation, proximity filtering, Redis caching, and low-latency API design.

2. Project Assumptions and Objectives

The main objective of this project was to add a restaurant geolocation module without breaking the existing versioned API. The new endpoints had to work under the existing student-specific prefix: `/api/78709/v1`.

- Verify that the current Docker-based system still works.
- Create a new restaurants table in PostgreSQL.
- Create a Restaurant model for Eloquent database interaction.
- Prepare a RestaurantSeeder with at least 10 restaurants, different coordinates, categories, ratings, and the album number 78709.
- Implement CRUD endpoints for restaurants.
- Implement a nearby search endpoint using latitude, longitude, and radius query parameters.
- Calculate distance in kilometers using the Haversine formula.
- Filter restaurants inside the selected radius and sort them by distance.
- Cache nearby search results in Redis for 60 seconds.
- Register routes correctly, with the nearby route before the restaurant resource route.
- Verify the API with cURL and save the final changes in Git.

3. Technological Foundations

PHP: The server-side programming language used for the backend logic.

Laravel: The backend framework used for routing, controllers, validation, Eloquent ORM, migrations, seeders, and JSON responses.

PostgreSQL: The relational database used to store restaurants and their coordinates.

Redis: The in-memory cache used to store repeated nearby search results.

Docker Compose: The tool used to run the backend, frontend, PostgreSQL, and Redis containers.

Nginx: The reverse proxy used to expose the application through port 8080.

Haversine Formula: A mathematical formula used to calculate approximate distance between two points on the Earth using latitude and longitude.

Git: The version control system used to commit the implementation changes.

cURL: The command-line tool used to test endpoints and verify status codes.

4. Implementation Process

4.1 Project Implementation Screenshots

The following screenshots document the implementation and verification process for Project 7. They show both code-level changes and terminal test results.

1. Baseline Docker Compose Verification

Before adding the new module, Docker services and existing API endpoints were verified. The screenshot shows running containers and working /api/health and /api/78709/v1/tasks endpoints.

```
jad@fedora-2:~/uni-project/project7$ docker compose ps
NAME                  IMAGE                  SERVICE         STATUS                  PORTS
wsd_backend           wsd-project-backend     backend         Up 8 minutes            0.0.0.0:8000->8000/tcp
wsd_db                postgres:16-alpine      db              Up 8 minutes            5432/tcp
wsd_redis             redis:7-alpine          redis           Up 8 minutes            6379/tcp
wsd_web               wsd-project-web         web             Up 8 minutes            0.0.0.0:8080->80/tcp

jad@fedora-2:~/uni-project/project7$ curl http://127.0.0.1:8080/api/health
{"status":"ok","service":"laravel-api","version":"v1"}

jad@fedora-2:~/uni-project/project7$ curl http://127.0.0.1:8080/api/78709/v1/tasks
[{"id":1,"title":"Prepare cache design note","status":"done","album_number":"78709"}]
```

Screenshot 1: Baseline Docker Compose and existing API verification.

2. File Generation Commands

The migration, model, seeder, and controller were generated for the restaurant module. These files became the basis for the new proximity search feature.

```
jad@fedora-2:~/uni-project/project7/backend$ php artisan make:migration create_restaurants_table
INFO Migration [database/migrations/2026_08_30_230000_create_restaurants_table.php] created successfully.

jad@fedora-2:~/uni-project/project7/backend$ php artisan make:model Restaurant
INFO Model [app/Models/Restaurant.php] created successfully.

jad@fedora-2:~/uni-project/project7/backend$ php artisan make:seeder RestaurantSeeder
INFO Seeder [database/seeders/RestaurantSeeder.php] created successfully.

jad@fedora-2:~/uni-project/project7/backend$ php artisan make:controller Api/RestaurantController
INFO Controller [app/Http/Controllers/Api/RestaurantController.php] created successfully.
```

Screenshot 2: Laravel file generation commands for Project 7.

3. Docker Rebuild

After code changes, the backend and web containers were rebuilt so the running application used the latest implementation.

```
jad@fedora-2:~/uni-project/project7$ docker compose up -d --build
✓ Image wsd-project-backend   Built                                    12.4s
✓ Image wsd-project-web       Built                                    12.4s
✓ Container wsd_redis          Running                                 0.0s
✓ Container wsd_db             Running                                 0.0s
✓ Container wsd_backend        Started                                 2.6s
✓ Container wsd_web            Started                                 0.9s
```

Screenshot 3: Docker rebuild for backend and web containers.

4. Route List Verification

The Laravel route list confirms that restaurant CRUD routes and the restaurants/nearby route were registered successfully.

```
jad@fedora-2:~/uni-project/project7/backend$ php artisan route:list
GET|HEAD   api/78709/v1/restaurants ..................... restaurants.index › Api\RestaurantController@index
POST       api/78709/v1/restaurants ..................... restaurants.store › Api\RestaurantController@store
GET|HEAD   api/78709/v1/restaurants/nearby .............. Api\RestaurantController@nearby
GET|HEAD   api/78709/v1/restaurants/{restaurant} ........ restaurants.show › Api\RestaurantController@show
PUT|PATCH  api/78709/v1/restaurants/{restaurant} ........ restaurants.update › Api\RestaurantController@update
DELETE     api/78709/v1/restaurants/{restaurant} ........ restaurants.destroy › Api\RestaurantController@destroy
Showing [39] routes
```

Screenshot 4: Route list showing restaurant and nearby endpoints.

5. Migration File for restaurants Table

The migration creates the restaurants table with name, latitude, longitude, category, rating, album_number, and timestamps.

2026_08_30_230000_create_restaurants_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('restaurants', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->decimal('latitude', 10, 7);
            $table->decimal('longitude', 10, 7);
            $table->string('category')->nullable();
            $table->float('rating')->default(0);
            $table->string('album_number');
            $table->timestamps();
        });
    }
};
```

Screenshot 5: Migration file for restaurants table.

6. Restaurant Model

The Restaurant model defines fillable fields for safe Eloquent mass assignment.

Restaurant.php backend > app > Models > Restaurant.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Restaurant extends Model
{
    protected $fillable = [
        'name',
        'latitude',
        'longitude',
        'category',
        'rating',
        'album_number',
    ];
}
```

Screenshot 6: Restaurant model with fillable fields.

7. RestaurantSeeder - Sample Data

The RestaurantSeeder inserts deterministic dummy data. The records include different cities, categories, ratings, coordinates, and the student album number 78709.

RestaurantSeeder.php

```php
        Restaurant::insert([
            [
                'name' => 'Katowice Pizza House',
                'latitude' => 50.2649000,
                'longitude' => 19.0238000,
                'category' => 'pizza',
                'rating' => 4.5,
                'album_number' => '78709',
                'created_at' => now(),
                'updated_at' => now(),
            ],
            [
                'name' => 'Sushi Katowice Center',
                'latitude' => 50.2599000,
                'longitude' => 19.0216000,
                'category' => 'sushi',
                'rating' => 4.7,
                'album_number' => '78709',
                'created_at' => now(),
                'updated_at' => now(),
            ],
```

Screenshot 7a: RestaurantSeeder with Katowice sample records.

8. RestaurantSeeder - More Records

Additional restaurants were added for Warsaw and Krakow. This gives the nearby endpoint enough data to filter and sort by distance.

RestaurantSeeder.php

```php
            [
                'name' => 'Warsaw Pizza Market',
                'latitude' => 52.2297000,
                'longitude' => 21.0122000,
                'category' => 'pizza',
                'rating' => 4.4,
                'album_number' => '78709',
                'created_at' => now(),
                'updated_at' => now(),
            ],
            [
                'name' => 'Krakow Sushi Bar',
                'latitude' => 50.0647000,
                'longitude' => 19.9450000,
                'category' => 'sushi',
                'rating' => 4.8,
                'album_number' => '78709',
                'created_at' => now(),
                'updated_at' => now(),
            ],
```

Screenshot 7b: RestaurantSeeder with Warsaw and Krakow records.

9. RestaurantSeeder - Final Records

The seeder also contains Gdansk restaurants. This makes the dataset geographically diverse.

RestaurantSeeder.php

```
95              [
96                  'name' => 'Gdansk Sea Pizza',
97                  'latitude' => 54.3520000,
98                  'longitude' => 18.6466000,
99                  'category' => 'pizza',
100                 'rating' => 4.0,
101                 'album_number' => '78709',
102                 'created_at' => now(),
103                 'updated_at' => now(),
104             ],
```

Screenshot 7c: RestaurantSeeder with final records.

10. DatabaseSeeder Update

The DatabaseSeeder was updated to call RestaurantSeeder, so restaurant records are inserted during seeding.

DatabaseSeeder.php — backend > database > seeders > DatabaseSeeder.php

```
11      public function run(): void
12      {
13          $this->call([
14              RestaurantSeeder::class,
15          ]);
16      }
17
18
```

Screenshot 8: DatabaseSeeder calling RestaurantSeeder.

11. RestaurantController - CRUD Methods

The controller implements index, store, show, update, and destroy methods for restaurant CRUD operations.

RestaurantController.php — app/Http/Controllers/Api/RestaurantController.php

```php
    public function store(Request $request): JsonResponse
    {
        $validated = $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'latitude' => ['required', 'numeric', 'between:-90,90'],
            'longitude' => ['required', 'numeric', 'between:-180,180'],
            'category' => ['nullable', 'string', 'max:100'],
            'rating' => ['nullable', 'numeric', 'between:0,5'],
            'album_number' => ['required', 'string', 'max:50'],
        ]);

        $restaurant = Restaurant::create($validated);
        Cache::flush();

        return response()->json(['data' => $restaurant], 201);
    }


```

Screenshot 9: RestaurantController CRUD methods.

12. RestaurantController - Nearby Search

The nearby method validates lat, lng, and radius, builds a cache key, calculates distances, filters results inside the radius, and sorts by distance.

RestaurantController.php

```php
    public function nearby(Request $request): JsonResponse
    {
        $validated = $request->validate([
            'lat' => ['required', 'numeric', 'between:-90,90'],
            'lng' => ['required', 'numeric', 'between:-180,180'],
            'radius' => ['required', 'numeric', 'min:0.1', 'max:100'],
        ]);

        $lat = (float) $validated['lat'];
        $lng = (float) $validated['lng'];
        $radius = (float) $validated['radius'];

        $cacheKey = "nearby:{$lat}:{$lng}:{$radius}";
```

Screenshot 10: Nearby search method with Redis Cache::remember.

13. RestaurantController - Haversine Calculation

The private calculateDistance method implements the Haversine formula and returns distance in kilometers.

RestaurantController.php

```php
private function calculateDistance(float $lat1, float $lng1, float $lat2, float $lng2): float
{
    $earthRadiusKm = 6371;

    $lat1Rad = deg2rad($lat1);
    $lng1Rad = deg2rad($lng1);
    $lat2Rad = deg2rad($lat2);
    $lng2Rad = deg2rad($lng2);

    $latDiff = $lat2Rad - $lat1Rad;
    $lngDiff = $lng2Rad - $lng1Rad;

    $a = sin($latDiff / 2) ** 2
        + cos($lat1Rad) * cos($lat2Rad) * sin($lngDiff / 2) ** 2;

    $c = 2 * atan2(sqrt($a), sqrt(1 - $a));

    return round($earthRadiusKm * $c, 3);
}
```

Screenshot 11: Haversine distance calculation method.

14. API Routes File

The restaurants/nearby route is placed before Route::apiResource for restaurants. This order is important because otherwise Laravel may treat nearby as a restaurant ID.

api.php — backend > routes > api.php

```php
Route::get('/restaurants/nearby', [RestaurantController::class, 'nearby']);
Route::apiResource('restaurants', RestaurantController::class);
```

Screenshot 12: api.php routes with nearby before apiResource.

15. Successful GET /restaurants Request

The restaurants list endpoint returned HTTP 200 OK and JSON data containing the seeded restaurants.

```
jad@fedora-2:~/uni-project/project7$ curl -i http://127.0.0.1:8080/api/78709/v1/restaurants
HTTP/1.1 200 OK
Server: nginx/1.29.8
Content-Type: application/json

{"data":[{"id":1,"name":"Katowice Pizza
House","latitude":"50.2649000","longitude":"19.0238000","category":"pizza","rating":4.5,"album_number":"78709"},
{"id":2,"name":"Sushi Katowice
Center","latitude":"50.2599000","longitude":"19.0216000","category":"sushi","rating":4.7,"album_number":"78709"}]}
```

Screenshot 13: Successful GET /api/78709/v1/restaurants request.

16. Successful POST /restaurants Request

The POST endpoint created a new restaurant and returned HTTP 201 Created.

```
jad@fedora-2:~/uni-project/project7$ curl -i -X POST http://127.0.0.1:8080/api/78709/v1/restaurants -H "Content-
Type: application/json" -d '{"name":"Test
Restaurant","latitude":50.2649,"longitude":19.0238,"category":"pizza","rating":4.6,"album_number":"78709"}'
HTTP/1.1 201 Created
Content-Type: application/json

{"data":{"name":"Test
Restaurant","latitude":50.2649,"longitude":19.0238,"category":"pizza","rating":4.6,"album_number":"78709","id":11}}
```

Screenshot 14: Successful POST /api/78709/v1/restaurants request.

17. Successful Nearby Search Request

The nearby endpoint returned HTTP 200 OK, the query parameters, a count, and restaurants sorted by distance_km.

```
jad@fedora-2:~/uni-project/project7$ curl -i "http://127.0.0.1:8080/api/78709/v1/restaurants/nearby?
lat=50.2649&lng=19.0238&radius=10"
HTTP/1.1 200 OK
Content-Type: application/json

{"query":{"latitude":50.2649,"longitude":19.0238,"radius_km":10},"count":4,"data":[{"id":1,"name":"Katowice Pizza
House","latitude":"50.2649000","longitude":"19.0238000","distance_km":0},{"id":2,"name":"Sushi Katowice
Center","distance_km":0.578},{"id":3,"name":"Vegan Bistro Katowice","distance_km":0.736},{"id":4,"name":"Burger
Point Katowice","distance_km":1.574}]}
```

Screenshot 15: Successful nearby search request.

18. Redis Cache Verification

Redis was checked before and after calling the nearby endpoint. Redis PING also returned PONG, confirming that Redis was reachable.

```
jad@fedora-2:~/uni-project/project7$ docker compose exec redis redis-cli DBSIZE
(integer) 0
jad@fedora-2:~/uni-project/project7$ curl "http://127.0.0.1:8080/api/78709/v1/restaurants/nearby?
lat=50.2649&lng=19.0238&radius=10"
{"count":4,"data":[...]}
jad@fedora-2:~/uni-project/project7$ docker compose exec redis redis-cli DBSIZE
(integer) 1
jad@fedora-2:~/uni-project/project7$ docker compose exec redis redis-cli PING
PONG
```

Screenshot 16: Redis DBSIZE and PING after nearby query testing.

19. Git Commit

The final Project 7 changes were committed to Git with a descriptive commit message.

```
jad@fedora-2:~/uni-project/project7$ git commit -m "Project 7: add restaurant proximity search"
[master f43a038] Project 7: add restaurant proximity search
6 files changed, 315 insertions(+)
create mode 100644 backend/app/Http/Controllers/Api/RestaurantController.php
create mode 100644 backend/app/Models/Restaurant.php
create mode 100644 backend/database/migrations/2026_08_30_230000_create_restaurants_table.php
create mode 100644 backend/database/seeders/RestaurantSeeder.php
```

Screenshot 17: Git commit for Project 7.

4.2 Conceptual Explanations

What Proximity Search Means:
Proximity search means finding objects that are physically close to a given location. In this project, restaurants have latitude and longitude values, and the user provides current coordinates with a radius. The backend calculates which restaurants are inside the selected radius and returns them as JSON.

Why Distance Calculation Is Necessary:
A normal CRUD API usually works with IDs and database rows. A proximity system is more complex because it must reason about geography. The backend cannot only compare text or identifiers; it must calculate physical distance between two coordinate points. This is why the Haversine formula was implemented in the controller.

What the Haversine Formula Does:
The Haversine formula estimates the distance between two points on the surface of the Earth using latitude and longitude. It converts degrees into radians, compares the two positions, and multiplies the result by the Earth radius. In this project, the returned value is rounded to kilometers and stored as distance_km in the API response.

Why Nearby Results Are Sorted by Distance:
Sorting by distance makes the response more useful. A user who searches for restaurants nearby expects the closest restaurants first. This also follows common behavior in food delivery, maps, ride-hailing, and recommendation systems.

Why Redis Cache Is Useful:
Nearby search can become read-heavy because many users may repeatedly search from similar locations. Redis reduces repeated database work by storing the result of a nearby query for 60 seconds. The cache key contains latitude, longitude, and radius, for example nearby:50.2649:19.0238:10. This means the same query can be served quickly from memory.

Why Spatial Systems Are Difficult at Scale:
Spatial systems become difficult at scale because the system may need to search millions of objects while keeping latency low. A simple approach that calculates distance for every record works for laboratory data, but it is not efficient for very large datasets. Production systems need better indexing, caching, and partitioning strategies.

4.3 Troubleshooting Technical Issues

1. File Generation and Docker: During implementation, it was important to create Laravel files in the local backend folder so they appeared in VS Code and were included in Git. After editing the files, Docker containers were rebuilt so the running application used the updated code.

2. Route Order for restaurants/nearby: The custom restaurants/nearby route had to be placed before Route::apiResource(restaurants). If the resource route is placed first, Laravel may interpret nearby as a restaurant ID and route the request incorrectly.

3. Cache Invalidation: When restaurants are created, updated, or deleted, the controller calls Cache::flush(). This guarantees that old nearby results are cleared.

5. Conclusion

In this project, I successfully extended the existing Laravel web system with a restaurant proximity search module. The implementation added a new restaurants table, Restaurant model, RestaurantSeeder, RestaurantController, CRUD endpoints, nearby search endpoint, Haversine distance calculation, Redis caching, and versioned API routes.

The system can now store restaurants with latitude and longitude values and return nearby restaurants based on user coordinates and search radius. The response includes distance_km and is sorted by distance, making it useful for location-based applications.

6. References

1. Laravel Documentation - Routing: https://laravel.com/docs/routing
2. Laravel Documentation - Controllers: https://laravel.com/docs/controllers
3. Laravel Documentation - Eloquent ORM: https://laravel.com/docs/eloquent
4. Laravel Documentation - Validation: https://laravel.com/docs/validation
5. Laravel Documentation - Migrations: https://laravel.com/docs/migrations
6. Laravel Documentation - Cache: https://laravel.com/docs/cache
7. MDN Web Docs - HTTP Methods: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods
8. MDN Web Docs - HTTP Status Codes: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
9. Haversine Formula: https://en.wikipedia.org/wiki/Haversine_formula
10. PostgreSQL Geometric Types: https://www.postgresql.org/docs/current/datatype-geometric.html
11. Redis Documentation: https://redis.io/docs/latest/
12. Docker Compose Documentation: https://docs.docker.com/compose/

Project 7 – Git Repository & Version Control Verification

All project modules, backend controllers, models, database migrations, automated test suites, frontend components, and compiled assignment reports have been committed and synchronized with the remote GitHub repository **Jadtales/uni-projects** on branch `master`.

```
jad@fedora-2:~/uni-project$ git status
On branch master
Your branch is up to date with 'origin/master'.
nothing to commit, working tree clean

jad@fedora-2:~/uni-project$ git log -1 --oneline
96dbd38 (HEAD -> master, origin/master) - projects
```